

**Embodied AI
Lab Manual for Isaac Lab/Isaac Sim
Isaac Lab + Isaac Sim Setup / R1
Implementation
August 2026**

Prepared by
Mario Ortiz

Table of Contents

1	Objective	3
2	Learning to use Isaac Lab + Isaac Sim for Locomotion	3
2.1	System Requirements	3
2.2	File Structure	4
2.3	Install Git.....	4
2.4	Download Isaac Sim.....	5
2.5	Clone Isaac Lab from GitHub	5
2.6	Bridge Isaac Sim to Isaac Lab	6
2.7	Install Isaac Lab Extensions	6
2.8	Verify Isaac Lab Bridge to Isaac Sim	6
2.9	Verify Isaac Lab using a Simulation Scene.....	7
2.10	Open the G1 Locomotion Configuration Files	7
2.11	Test the G1 Rough-Terrain Training Pipeline.....	8
2.11.1	Verify the Rough-Terrain Training Pipeline	8
2.11.2	Play the Untrained Rough Policy	9
2.11.3	Run a High Iteration Rough-Terrain Training Session	10
2.12	Compare Early, Middle, and Final Rough-Terrain Checkpoints.....	11
2.13	Test the G1 Flat-Terrain Training Pipeline.....	12
3	Second Stage of Lab: R1 Implementation	14
3.1	Download the Unitree ROS Repository.....	14
3.2	Create the R1 Asset Folder in Isaac Lab	15
3.3	Launch Isaac Sim and Enable the URDF Importer Extension.....	15
3.4	Import the R1 URDF into Isaac Sim	16
3.5	Set the Import Options	17
3.6	Confirm the USD Export and Verify the R1 USD Path	18
3.7	Open the Exported R1 USD in Isaac Sim.....	18
4	References	19

1 Objective

The purpose of this lab is to introduce students to Isaac Lab and Isaac Sim as well as provide hands-on experience with humanoid locomotion tasks using the Unitree G1. Students will begin by learning how to install and configure Isaac Lab and Isaac Sim on their computers. They will then run existing locomotion tasks provided by Isaac Lab to gain an understanding of the overall reinforcement learning workflow and how locomotion policies are trained.

Finally, students will learn how to integrate their own humanoid robots into Isaac Lab. This process will primarily involve converting a robot description from URDF (Unified Robot Description Format) to USD (Universal Scene Description), the format used by Isaac Sim and Isaac Lab for simulation and training.

Important notes:

- Isaac Lab: NVIDIA's open-source, modular framework for robot learning. It provides tools and environments for developing, training, and evaluating robot learning policies, including reinforcement learning for locomotion.
- Isaac Sim: NVIDIA's robotics simulation platform built on the NVIDIA Omniverse framework. It provides a physics-based environment for simulating robots and testing their behavior before deploying them on physical hardware.
- Unitree Robotics: A Chinese technology company that designs and manufactures advanced quadrupedal and humanoid robots, including the Unitree G1 humanoid robot used in this lab.
- Reinforcement Learning (RL): A type of machine learning in which an autonomous agent learns to make decisions by interacting with the environment. The agent performs actions and receives rewards or penalties based on the outcomes of those actions. In robotics, reinforcement learning can be used to teach robots complex behaviors, such as walking, running, maintaining balance, and adapting to uneven terrain.

2 Learning to use Isaac Lab + Isaac Sim for Locomotion

2.1 System Requirements

Before installation, these system requirements should be accounted for on the user's device:

- Operating System: Windows 10/11
- GPU: NVIDIA RTX GPU recommended
- GPU Driver: Recent NVIDIA Production Driver
- Storage: Enough Space for Isaac Sim and Isaac Lab
- Software: Git, Visual Studio Code (or any other IDE), and Command Prompt/PowerShell

2.2 File Structure

```

C:\
├─ isaacsim
│   └─ Isaac Sim installation files
│
├─ isaacclab
│   ├── _isaac_sim
│   │   └─ symbolic link to C:\isaacsim
│   │
│   ├── isaacclab.bat
│   ├── scripts
│   ├── source
│   ├── apps
│   └─ logs
│       └─ rsl_r1
│           └─ training logs will be created here after running tasks
│
└─ unitree_ros
    └─ robots
        └─ r1_description
            ├── R1.urdf
            └─ meshes
  
```

2.3 Install Git

Git is needed to download the Isaac Lab repository.

Download Git from:

<https://git-scm.com/downloads>

After installation, verify Git from Power shell using the command below. The successful installation of Git should print a version number.

```
git --version
```

Note: Across these four labs, **Visual Studio Code** will be used as the primary source-code editor. However, you are welcome to use any source-code editor of your choice, provided it supports the tools and file formats required for the labs.

2.4 Download Isaac Sim

Navigate to the official **Isaac Sim installation page** and download the pre-built workstation package for windows.

Official Isaac Sim installation documentation:

<https://docs.isaacsim.omniverse.nvidia.com/>

For these projects, Isaac Sim 5.1 was used. NVIDIA provides versioned documentation for Isaac Sim, including the *Isaac Sim 5.1 workstation installation page*.

After downloading the Windows package, extract the Isaac Sim files to a simple, easily accessible directory. A recommended installation location is:

```
C:\isaacsim
```

Next, confirm that the folder contains files such as:

```
isaac-sim.bat
```

```
python.bat
```

```
kit/
```

```
exts/
```

```
apps/
```

The important file for Isaac Lab is

```
C:\isaacsim\python.bat
```

This is Isaac Sim's bundled Python interpreter.

2.5 Clone Isaac Lab from GitHub

Open PowerShell and move to the folder where the Isaac Lab repository will be stored. For this project, the Isaac Lab folder was located at

```
C:\IsaacLab
```

Clone the repository:

```
cd C:\IsaacLab
```

```
git clone https://github.com/isaac-sim/IsaacLab.git
```

```
cd IsaacLab
```

2.6 Bridge Isaac Sim to Isaac Lab

Isaac Lab needs to know the location Isaac Sim is installed. We will create a symbolic link named:

`_isaac_sim`

inside the Isaac Lab repository. This link points to the Isaac Sim installation. In other words,

`C:\IsaacLab\_isaac_sim`

acts as a bridge to

`C:\isaacsim`

Open Command Prompt/Windows Powershell as Administrator. Then run:

```
cd C:\IsaacLab
```

```
mklink /D _isaac_sim C:\isaacsim
```

The Isaac Lab folder should now contain:

`C:\IsaacLab\_isaac_sim`

We can now use Isaac Sim's bundled Python environment rather than the system-wide Python installation, ensuring that Isaac Lab uses the correct dependencies and configuration required for the simulation environment.

2.7 Install Isaac Lab Extensions

Go to the main Isaac Lab directory and install:

```
cd C:\IsaacLab
```

```
.\isaacclab.bat -install
```

Or:

```
.\isaacclab.bat -i
```

The `isaacclab.bat` file is a Windows helper script that runs Python through Isaac Sim and installs the required Isaac Lab extensions and dependencies.

2.8 Verify Isaac Lab Bridge to Isaac Sim

Run:

```
cd C:\IsaacLab
```

```
.\isaacclab.bat -p -c "import sys; print(sys.executable)"
```

You should expect to see:

`C:\IsaacLab\_isaac_sim\python.bat`

Or another path inside the `_isaac_sim` installation.

2.9 Verify Isaac Lab using a Simulation Scene

Run:

```
cd C:\IsaacLab
.\isaac.bat -p .\scripts\tutorials\00_sim\create_empty.py
```

If the installation was successful, Isaac Sim should launch and open an empty simulation scene. A graphical user interface (GUI) window should appear; please note that the initial launch may take some time.

2.10 Open the G1 Locomotion Configuration Files

Open:

```
cd
C:\IsaacLab\source\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\g1
ls
```

Then (assuming installed Visual Studio Code):

```
code rough_env_cfg.py
code flat_env_cfg.py
code agents\rsl_rl_ppo_cfg.py
code __init__.py
```

These files can also be opened directly (from the original IsaacLab folder)

```
code .\source\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\g1\rough_env_cfg.py
code .\source\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\g1\flat_env_cfg.py
code
.\source\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\g1\agents\rsl_rl_ppo_cfg.py
code .\source\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\g1\__init__.py
```

File Explanation:

`rough_env_cfg.py`: This is the primary configuration file for the rough-terrain locomotion environment. It defines the robot asset, terrain settings, height scanner configuration, reward terms, command ranges, event parameters, and termination conditions used for rough-terrain walking.

`flat_env_cfg.py`: This file defines the flat-terrain G1 locomotion environment. It inherits the configuration from the rough-terrain environment while removing terrain-specific features, such as height scanning, that are not necessary for flat-ground locomotion.

`agents\rsl_rl_ppo_cfg.py`: This file defines the PPO training configuration used by RSL-RL. It controls key training parameters, including:

- Number of steps per environment
- Maximum number of training iterations

- Model save interval
- Actor and critic network sizes
- Learning rate
- Number of PPO epochs
- Number of mini-batches
- Discount factor
- Entropy coefficient

`__init__.py`: This file registers the task names with Gymnasium. A task registration connects the task name with its corresponding environment configuration class and PPO runner configuration class.

The task name is then used when launching an environment. For example:

```
--task Isaac-Velocity-Rough_G1-v0
```

After creating the R1 task, the corresponding command would be:

```
--task Isaac-Velocity-Rough_r1-v0
```

Proper task registration is required for Isaac Lab to recognize and launch a custom environment using its assigned task name.

2.11 Test the G1 Rough-Terrain Training Pipeline

After reviewing the baseline G1 configuration files, the next step is to learn to use the rough-terrain training pipeline.

The rough-terrain G1 training task is:

```
Isaac-Velocity-Rough-G1-v0
```

The rough-terrain G1 play task is:

```
Isaac-Velocity-Rough-G1-Play-v0
```

The train script is:

```
scripts\reinforcement_learning\rsl_rl\train.py
```

The play script is:

```
scripts\reinforcement_learning\rsl_rl\play.py
```

2.11.1 Verify the Rough-Terrain Training Pipeline

We will verify that the rough-terrain G1 environment can successfully launch, spawn the robot, generate the terrain, initialize the neural network, and start the RSL-RL training loop. This verification is important to perform before beginning a full training run, as actual training can require a significant amount of time.

The purpose of this step is not to train a useful walking policy. Instead, it confirms that the entire training pipeline is configured correctly and functioning as expected before committing to a longer training session.

```
cd C:\IsaacLab
.\isaacclab.bat -p .\scripts\reinforcement_learning\rs1_r1\train.py `
    --task Isaac-Velocity-Rough-G1-v0 `
    --num_envs 4 `
    --max_iterations 1
```

After running, Isaac Lab should create a new log folder under:

C:\IsaacLab\logs\rs1_r1\g1_rough

Each training run creates a timestamped folder, for example:

2026-08-22_13-30-00

Inside this folder, there should be checkpoint files such as:

model_0.pt

or other model_*.pt files.

2.11.2 Play the Untrained Rough Policy

To observe the robot's behavior with essentially no training, run the following command to load and play the specified checkpoint:

```
.\isaacclab.bat -p .\scripts\reinforcement_learning\rs1_r1\play.py ` --task Isaac-Velocity-
Rough-G1-Play-v0 ` --num_envs 4 ` --checkpoint "$roughPath\model_0.pt"
```

At this stage, the robot should spawn in the correct environment but will likely fall almost immediately due to the lack of training. This behavior is expected. Successfully spawning the robot in the intended environment confirms that the environment, checkpoint, and training pipeline have been configured correctly.

You should also see the output text, containing information of the training per each iteration. Here is an example of one below:

```

#####
                        Learning iteration 1991/2000

                        Total steps: 48955392
                        Steps per second: 10750
                        Collection time: 2.024s
                        Learning time: 0.262s
                        Mean value loss: 0.0018
                        Mean surrogate loss: -0.0085
                        Mean entropy loss: 25.6636
                        Mean reward: 22.32
                        Mean episode length: 995.92
                        Mean action std: 0.75
Episode_Reward/track_lin_vel_xy_exp: 0.9128
Episode_Reward/track_ang_vel_z_exp: 0.6537
Episode_Reward/lin_vel_z_l2: -0.0059
Episode_Reward/ang_vel_xy_l2: -0.0149
Episode_Reward/dof_torques_l2: -0.0081
Episode_Reward/dof_acc_l2: -0.0125
Episode_Reward/action_rate_l2: -0.2437
Episode_Reward/feet_air_time: 0.0428
Episode_Reward/flat_orientation_l2: -0.0141
Episode_Reward/dof_pos_limits: -0.0045
Episode_Reward/termination_penalty: 0.0000
Episode_Reward/feet_slide: -0.0152
Episode_Reward/joint_deviation_hip: -0.0275
Episode_Reward/joint_deviation_arms: -0.0773
Episode_Reward/joint_deviation_fingers: -0.0538
Episode_Reward/joint_deviation_torso: -0.0073
Metrics/base_velocity/error_vel_xy: 0.2726
Metrics/base_velocity/error_vel_yaw: 0.6661
Episode_Termination/time_out: 0.9844
Episode_Termination/base_contact: 0.0156
-----
                        Iteration time: 2.29s
                        Time elapsed: 01:07:37
                        ETA: 00:00:16

```

Figure 1: Example of the expected terminal output during training (not the exact iteration shown in this task). The output provides information on rewards, penalties, and simulation performance, giving the user insight into how well the training is progressing over time.

2.11.3 Run a High Iteration Rough-Terrain Training Session

We can now run a longer training session in headless mode. Navigate to the Isaac Lab directory and execute the following command:

```
cd C:\IsaacLab .\isaac.bat -p .\scripts\reinforcement_learning\rs1_rl\train.py --task Isaac-Velocity-Rough-G1-v0 --num_envs 1024 --max_iterations 3000 --headless
```

The **--headless** option disables the graphical user interface (GUI), allowing the training process to run without rendering the simulation. This reduces computational overhead and can significantly improve training performance.

During training, Isaac Lab saves model checkpoints in:

```
C:\IsaacLab\logs\rs1_rl\g1_rough\<RUN_FOLDER>
```

The run folder will contain multiple checkpoint files, such as:

```
model_0.pt
model_50.pt
```

model_100.pt
model_500.pt
model_1000.pt
model_1500.pt
model_2000.pt
model_2500.pt
model_2999.pt

The exact checkpoint filenames will depend on the configured save interval and the total number of training iterations completed.

2.12 Compare Early, Middle, and Final Rough-Terrain Checkpoints

After training, identify the newest rough G1 run.

Play the first checkpoint:

```
.\isaacclab.bat -p .\scripts\reinforcement_learning\rs1_r1\play.py `
  --task Isaac-Velocity-Rough-G1-Play-v0 `
  --num_envs 32 `
  --checkpoint "$roughPath\model_0.pt"
```

Play a middle checkpoint. Replace the filename with an actual checkpoint from the folder:

```
.\isaacclab.bat -p .\scripts\reinforcement_learning\rs1_r1\play.py `
  --task Isaac-Velocity-Rough-G1-Play-v0 `
  --num_envs 32 `
  --checkpoint "$roughPath\model_1500.pt"
```

Play the final checkpoint. Replace the filename with the newest checkpoint in the folder:

```
.\isaacclab.bat -p .\scripts\reinforcement_learning\rs1_r1\play.py `
  --task Isaac-Velocity-Rough-G1-Play-v0 `
  --num_envs 32 `
  --checkpoint "$roughPath\model_2999.pt"
```

The expected comparison is:

- model_0.pt → no useful walking behavior
- middle model → partial walking or unstable walking
- final model → better command tracking and more stable locomotion

2.13 Test the G1 Flat-Terrain Training Pipeline

The same steps will be done with the `flat_env_cfg.py` now.

The flat-ground G1 Training task is:

Isaac-Velocity-Flat-G1-v0

The flat-ground G1 play task is:

Isaac-Velocity-Flat-G1-Play-v0

Do this command

```
cd C:\IsaacLab
.\isaacclab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
    --task Isaac-Velocity-Flat-G1-v0 `
    --num_envs 4 `
    --max_iterations 1
```

Obtain the run (Can be found in your `C:\IsaacLab\logs\rsl_rl\g1_rough\<RUN_FOLDER>` folder.

Play the Checkpoint

```
.\isaacclab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
    --task Isaac-Velocity-Flat-G1-Play-v0 `
    --num_envs 4 `
    --checkpoint "$flatPath\model_0.pt"
```

Run Longer Flat Environment Training

```
cd C:\IsaacLab .\isaacclab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
    --task Isaac-Velocity-Flat-G1-v0 `
    --num_envs 1024 `
    --max_iterations 1500 `
    --headless
```

Compare Early, Middle, and Final Flat-Environment Checkpoints

Early Checkpoint

```
.\isaacclab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
    --task Isaac-Velocity-Flat-G1-Play-v0 `
```

```
--num_envs 32 `
--checkpoint "$flatPath\model_0.pt"
```

Middle Checkpoint

```
.\isaacsim.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
--task Isaac-Velocity-Flat-G1-Play-v0 `
--num_envs 32 `
--checkpoint "$flatPath\model_750.pt"
```

Final Checkpoint

```
.\isaacsim.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
--task Isaac-Velocity-Flat-G1-Play-v0 `
--num_envs 4 `
--checkpoint "$flatPath\model_1499.pt"
```

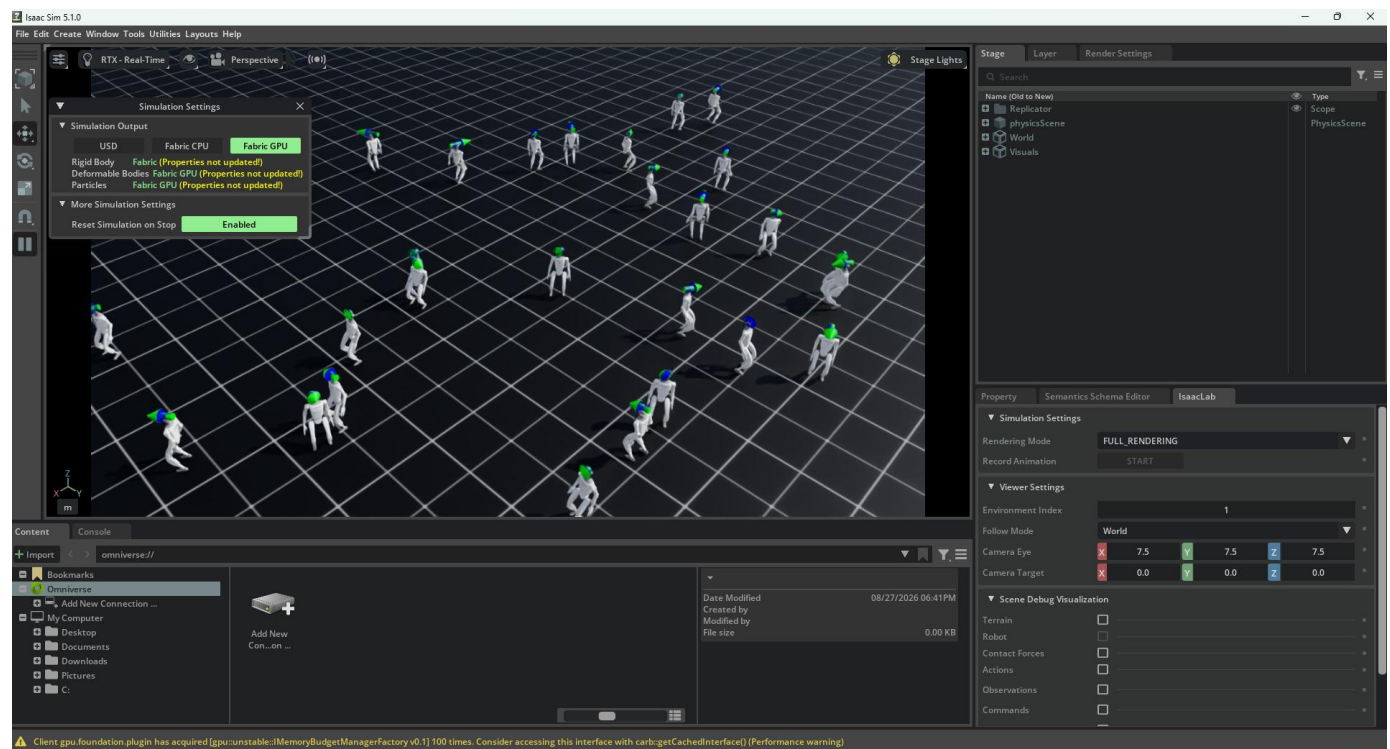


Figure 2: Example of the Isaac Sim GUI showing the training model developed with Isaac Lab for the Unitree G1 Flat-Terrain Task.



Figure 3: Closeup example of the Unitree G1 Flat-Terrain Locomotion Task in the Isaac Sim GUI.

3 Second Stage of Lab: R1 Implementation

Up to this point, we have been using the Unitree G1 along with the resources and environments provided by NVIDIA and Unitree. We will now extend this workflow to learn how to use Isaac Lab for reinforcement with a custom robot.

For this lab, we will use the **Unitree R1**. In future labs, we will build additional locomotion training protocols for the R1, which is not currently provided as a standard Isaac Lab training environment.

Isaac Lab environments use **USD (Universal Scene Description)** assets for robot simulation. The Unitree R1 robot description is available as a **URDF** model through Unitree's public **unitree_ros** repository. The repository provides robot descriptions for ROS1 and ROS2 and includes an **r1_description** directory containing the **R1.urdf** file and associated mesh files.

Because Isaac Lab uses USD assets, the R1 URDF must be converted into a USD asset to be integrated into our custom Isaac Lab environment.

3.1 Download the Unitree ROS Repository

Open PowerShell and move it to a convenient folder and download the Unitree repository.

Run:

```
cd C:
```

```
git clone https://github.com/unitreerobotics/unitree_ros.git
```

Then, open the R1 description folder:

```
cd C:\unitree_ros\robots\r1_description  
ls
```

The folder should contain:

```
R1.urdf  
Meshes
```

3.2 Create the R1 Asset Folder in Isaac Lab

The R1 USD asset must be stored inside a custom R1 task folder. Create this directory in:

```
cd C:\IsaacLab  
mkdir  
.\source\isaacclab_tasks\isaacclab_tasks\manager_based\locomotion\velocity\config\r1\assets\R1
```

This is the target export folder. The final USD file should reside in

```
C:\IsaacLab\source\isaacclab_tasks\isaacclab_tasks\manager_based\locomotion\velocity\config\r1\assets\R1\R1.usd
```

because the later R1 asset configuration file, `r1_asset_cfg.py` will point to this USD file.

3.3 Launch Isaac Sim and Enable the URDF Importer Extension

Run this command to open Isaac Sim

```
cd C:\IsaacLab  
.\isaacclab.bat -s
```

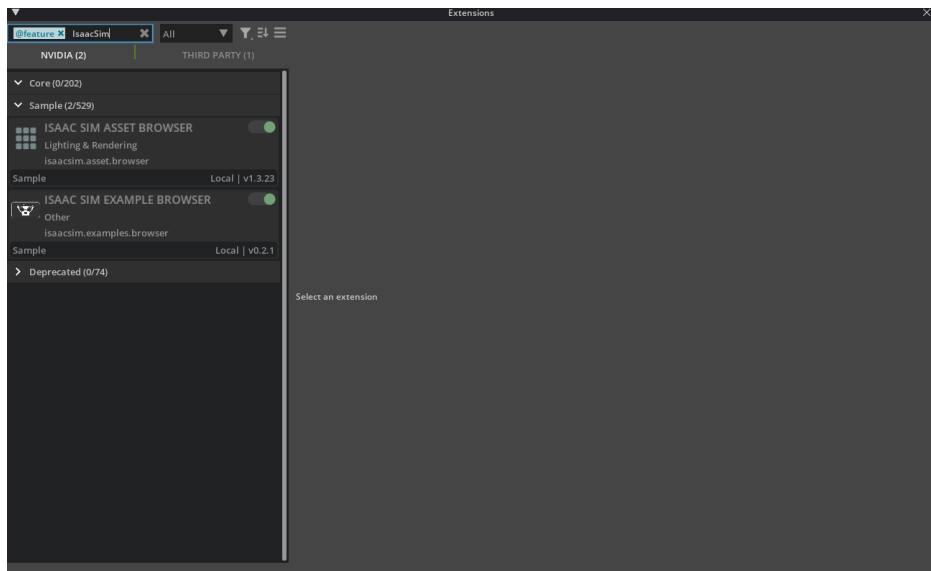
Once Isaac Sim opens, go to

Window > Extensions

Search for:

```
Isaacsim.asset.import.urdf
```

Or `Isaacsim.asset.browser`

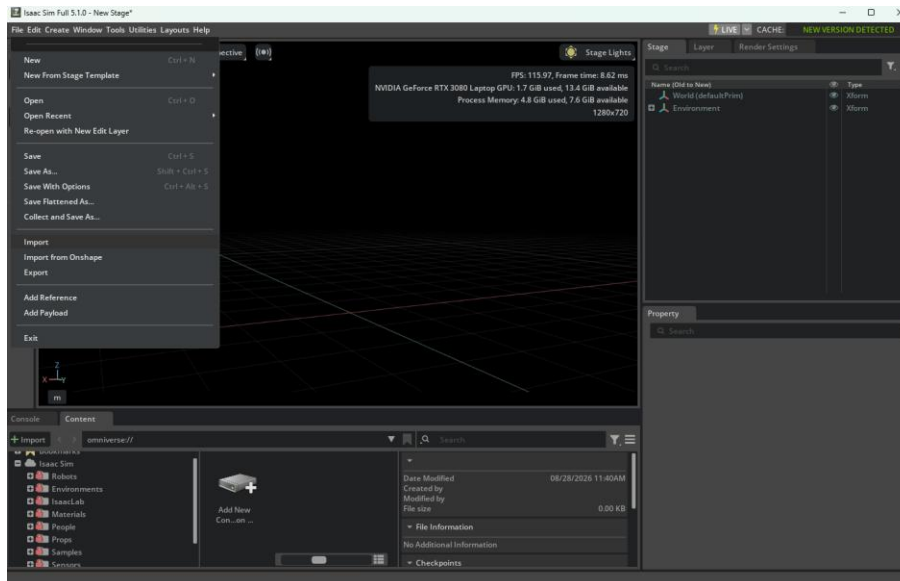


Enable the extension if it is not already enabled.

3.4 Import the R1 URDF into Isaac Sim

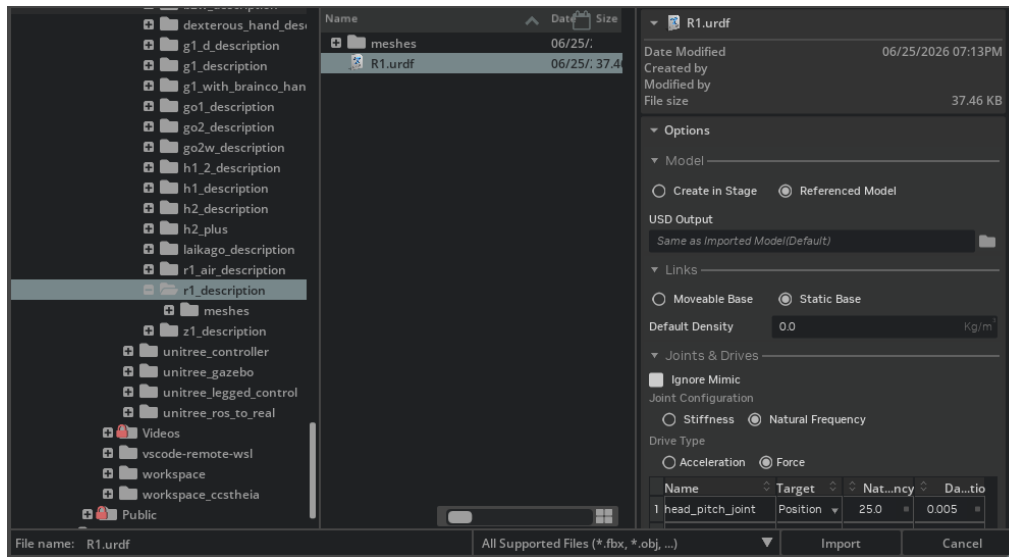
In Isaac Sim, go to:

File > Import



Select the R1 URDF file:

C:\unitree_ros\robots\r1_description\R1.urdf



This will convert the URDf file into a USD.

3.5 Set the Import Options

In the URDF import window, use these settings which are for a floating-based humanoid robot.

Model: Referenced Model

USD Output:

C:\isaacLab\source\isaacLab_tasks\isaacLab_tasks\manager_based\locomotion\velocity\config\r1\assets\R1

Links: Moveable Base

Default Density: 0.0 kg/m^3

Joints & Drives:

- Ignore Mimic: Unchecked

Drive Type: Force

Natural Frequency: 25.0

Damping Ratio: 0.005

Colliders:

- Collision From Visuals: Unchecked

Collider Type: Convex Hull

Allow Self-Collision: Unchecked

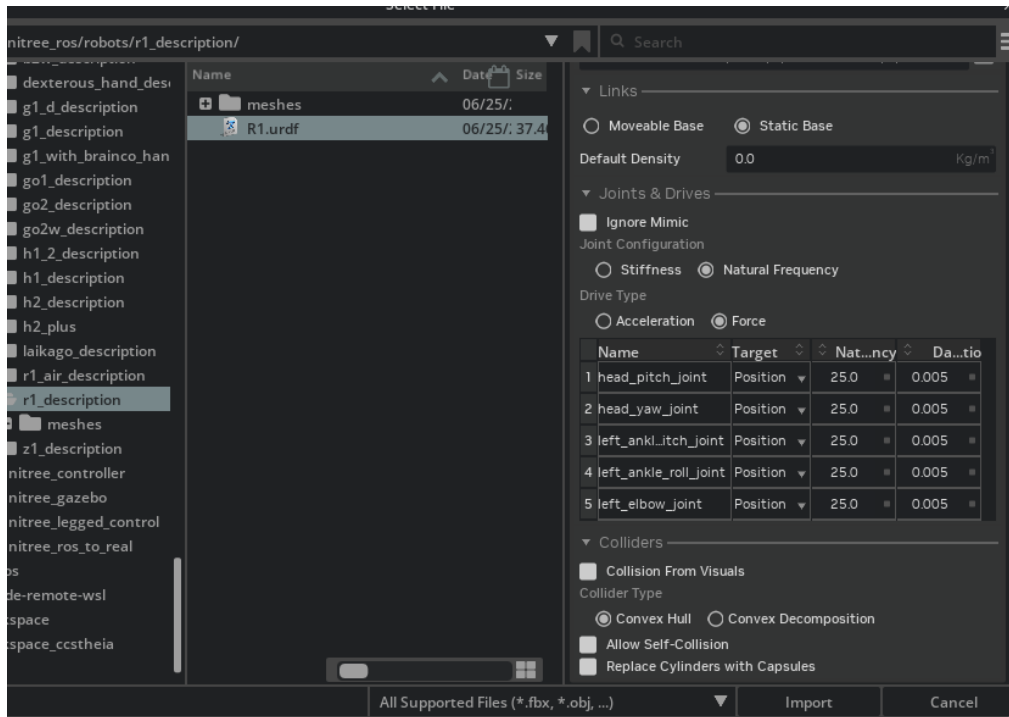
Replace Cylinders with Capsules: Unchecked

For the USD output location, set the output folder to:

C:\IsaacLab\source\isaacLab_tasks\isaacLab_tasks\manager_based\locomotion\velocity\config\r1\assets\R1

Then click:

Import



3.6 Confirm the USD Export and Verify the R1 USD Path

After importing, confirm that the USD file was created. In PowerShell, run:

```
cd C:\IsaacLab
ls
.\source\isaac\lab_tasks\isaac\lab_tasks\manager_based\locomotion\velocity\config\r1\assets\R1\
```

This folder should contain:

```
R1.usd
configuration
```

The *configuration* folder may contain supporting USD files and mesh references created during the import process. *R1.usd* is the converted USD file.

The final asset path should be:

```
C:\IsaacLab\source\isaac\lab_tasks\isaac\lab_tasks\manager_based\locomotion\velocity\config\r1\assets\R1\R1.usd
```

3.7 Open the Exported R1 USD in Isaac Sim

This will verify that the robot asset was created correctly. From PowerShell, run:

```
cd C:\IsaacLab
```

```
.\isaacsim.bat -s
```

Then at Isaac Sim, go to:

File > Open

Then select the file:

```
C:\IsaacLab\source\isaacsim_tasks\isaacsim_tasks\manager_based\locomotion\velocity\config\r1\assets\R1\R1.usd
```

This should open the R1 USD as the active stage.

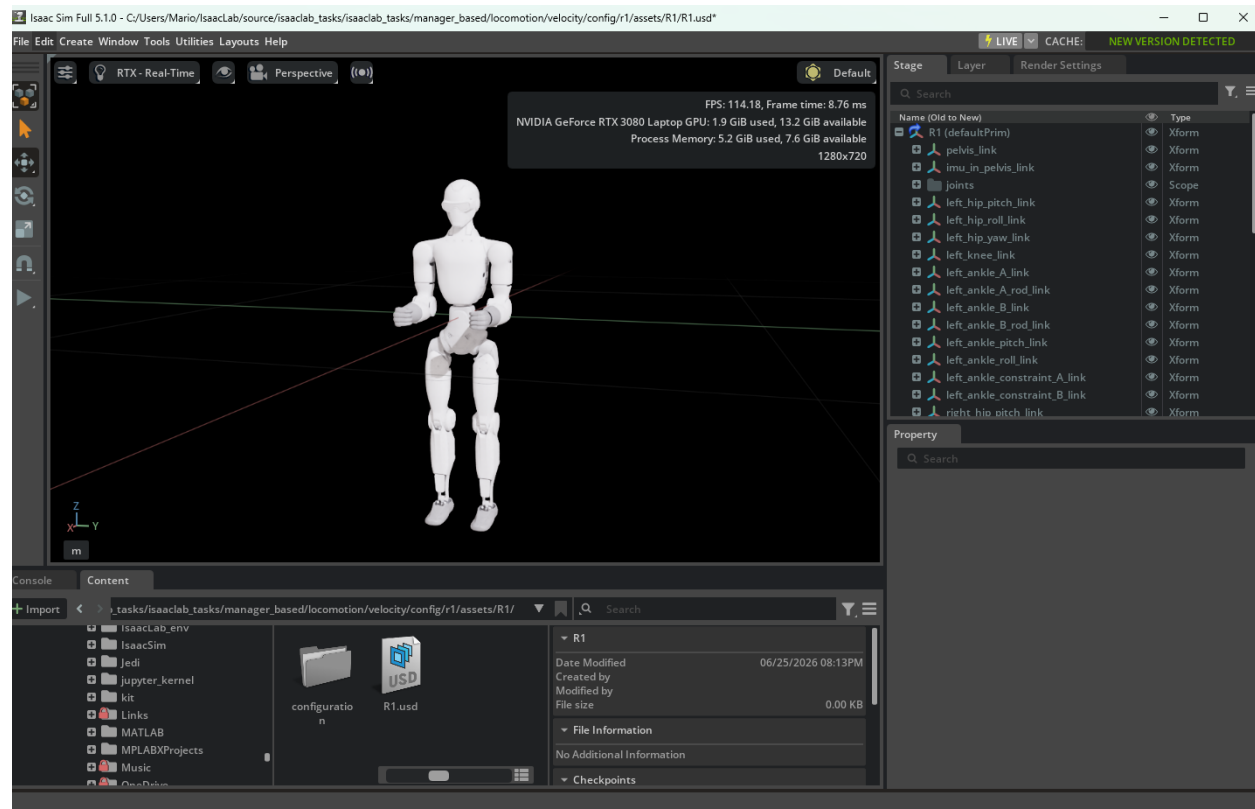


Figure 4: Isaac Sim GUI showcasing the Unitree R1 USD asset

4 References

- [1] NVIDIA, “Installation using Isaac Sim Pre-built Binaries,” Isaac Lab Documentation
- [2] NVIDIA, “URDF Importer Extension,” Isaac Sim Documentation.
- [3] NVIDIA, “Tutorial: Import URDF,” Isaac Sim Documentation.
- [4] Unitree Robotics, “unitree_ros,” GitHub repository.
- [5] Git, “Git Downloads.”